OBJECT ORIENTED SOFTWARE ENGINEERING

Ms. Archana A

Department of Computer Applications

OBJECT ORIENTED SOFTWARE ENGINEERING

Design Concepts…

Archana A
Department of Computer Applications

The Design Concepts

Design concepts provide the **necessary framework** "**to get the thing on right way**".

1. Abstraction.
2. Architecture.
3. Patterns.
4. **Separation of concerns.**
5. **Modularity**
6. **Information Hiding.**
7. Functional independence.
8. Refinement
9. Aspects
10. Refactoring.
11. Object Oriented design concepts
12. Design Classes.

4. Separation of Concern

- Any **complex problem** can be more **easily handled** if it is **subdivided** into pieces that each can be solved and/or optimized independently.

- A ***concern*** is a **feature** or **behavior** that is specified as **part of the requirements model** for the software.

- By **separating concerns** into smaller, and more **manageable pieces**, a problem takes **less** **effort and time** to solve.

4. Separation of Concern…

- This leads to **divide-and-conquer** strategy—it's easier to solve a **complex problem** when you **break it into manageable pieces.** This has important implications with regard to **software modularity.**

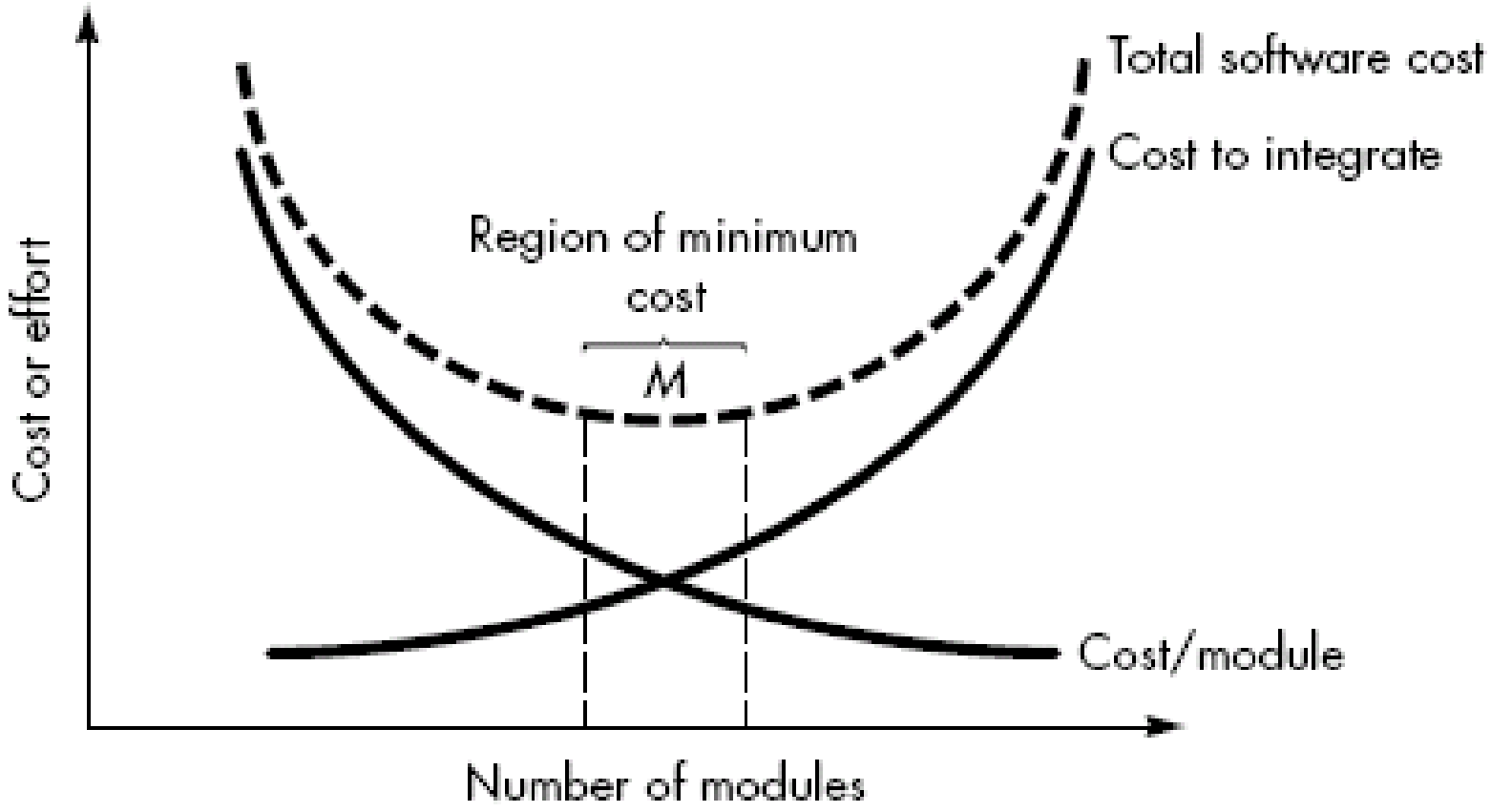
- Separation of concerns is manifested in other related design concepts:
- **modularity,**
- **aspects,**
- **functional independence, and**
- **refinement**.

5. Modularity

- Software is divided into separately **named** and **addressable components,** called **modules,** which are **integrated** to satisfy problem requirements.

- modularity is a single attribute of software that allows a program to be **intellectually manageable**.

Modularity and Software Cost



5. Modularity

- Refer fig. that state that **effort (cost) to develop an individual software module does decrease if total number of modules are less.**

- However as the **number of modules grows, the effort (cost) associated with integrating the modules also grows**.

5. Modularity…

- **Undermodularity and overmodularity should be avoided.**
  - We modularize a design so that development can be more **easily planned**.
  - **Software increments** can be defined and delivered.
  - **Changes** can be more easily accommodated.
  - **Testing and debugging** can be conducted more **efficiently** and long-term maintenance can be conducted without serious side effects.

5. Modularity…

The importance of modularity in software engineering can be highlighted in several ways:

1. Ease of Development:

Modularity allows software developers to work on individual modules independently, **without interfering with other parts** of the system.

- This division of labor makes the development process more manageable and enables parallel development, which can significantly **reduce the time required to build complex software** systems.

2. Enhanced Maintainability:

Modular software is **easier to maintain and update** because **changes made to one module are less likely to impact other modules**.

- This makes it easier to **fix bugs, add new features**, or **improve existing functionality without causing unintended side effects** throughout the system.
- It also facilitates **code reuse**, as modular components can be easily integrated into other projects or future versions of the software.

3. Facilitates Testing and Debugging:

Modular software is easier to test and debug because each **module can be tested independently** of the rest of the system.

- This isolates potential issues and simplifies the debugging process, making it **faster and more efficient** to identify and fix problems.

4.Encourages Collaboration:

Modularity encourages collaboration among development teams by providing clear boundaries between different components of the system.

- This enables teams to work on **separate modules simultaneously**, **share resources** **and expertise**, **and** **integrate their contributions** more seamlessly..

6. Information Hiding

- The principle of ***information hiding*** suggests that "**modules be characterized by design decisions that (each) hides from all others**."

- **i.e., Modules** should be **specified** and **designed** so that information (algorithm and data) contained within a **module is inaccessible to other modules** that have no need of such information.

6. Information Hiding

- The intent of information hiding is to **hide the details of data structure and procedural** processing behind a module interface. **Knowledge of the details need not be known by the users of module**

- **Enhanced Security and Confidentiality:** Information hiding helps improve security and confidentiality by **limiting access to sensitive or critical data** and **functionality.**

- By exposing only necessary interfaces, modules can enforce access controls and prevent unauthorized manipulation of internal state and behavior. This helps mitigate security risks and protect sensitive information from malicious attacks or unintended misuse.

6. Information Hiding

- Information hiding gives **benefits** when **modifications are required during testing and maintenance,** because data and procedure are hidden from other parts of software, **unintentional errors** introduced during modification, **are less**.

THANK YOU

Ms. Archana A
Department of Computer Applications

archana@pes.edu

+91 80 6666 3333 Extn 392